

Ficheros



Aunque los ficheros encajarían más en un apartado de «*entrada/salida*» ya que representan un **medio de almacenamiento persistente**, también podrían ser vistos como *estructuras de datos*, puesto que nos permiten guardar la información y asignarles un cierto formato. ¹

Un **fichero** es un *conjunto de bytes* almacenados en algún *dispositivo*. El [sistema de ficheros](#) es la estructura lógica que alberga los ficheros y está jerarquizado a través de *directorios* (o carpetas). **Cada fichero se identifica unívocamente a través de una ruta que nos permite acceder a él.**

Lectura de un fichero

Python ofrece la función `open()` para «abrir» un fichero. Esta apertura se puede realizar en 3 modos distintos:

- **Lectura** del contenido de un fichero existente.
- **Escritura** del contenido en un fichero nuevo.
- **Añadido** al contenido de un fichero existente.

Veamos un ejemplo para leer el contenido de un fichero en el que se encuentran las temperaturas mínimas y máximas de cada día de la última semana. El fichero está en la subcarpeta (*ruta relativa*) `files/temps.dat` y tiene el siguiente contenido:

```
23 29
23 31
26 34
23 33
22 29
22 28
22 28
```

Lo primero será abrir el fichero:

```
>>> f = open('files/temps.dat', 'r')
```

>>>

La función `open()` recibe como primer argumento la **ruta al fichero** que queremos manejar (como un «string») y como segundo argumento el **modo de apertura** (también como un «string»). Nos **devuelve el manejador del fichero**, que en este caso lo estamos asignando a una variable llamada `f` pero le podríamos haber puesto cualquier otro nombre.

Nota

Es importante dominar los conceptos de **ruta relativa** y **ruta absoluta** para el trabajo con ficheros. Véase [este artículo de DeNovatoANovato](#).

El **manejador del fichero** se implementa mediante un [flujo de entrada/salida](#) para las operaciones de lectura/escritura. Este objeto almacena, entre otras cosas, la *ruta al fichero*, el *modo de apertura* y la *codificación*:

```
>>> f
<_io.TextIOWrapper name='files/temps.dat' mode='r' encoding='UTF-8'>
```

>>>

Truco

Existen muchas [codificaciones de caracteres](#) para ficheros, pero la más utilizada es [UTF-8](#) ya que es capaz de representar cualquier carácter [Unicode](#) al utilizar una longitud variable de 1 a 4 bytes.

Hay que tener en cuenta que la ruta al fichero que abrimos (*en modo lectura*) **debe existir**, ya que de lo contrario obtendremos un error:

```
>>> f = open('foo.txt', 'r')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [Errno 2] No such file or directory: 'foo.txt'
```

>>>

Una vez abierto el fichero ya podemos proceder a leer su contenido. Para ello Python nos ofrece la posibilidad de leer todo el fichero de una vez o bien leerlo línea a línea.

Lectura completa de un fichero

Siguiendo con nuestro ejemplo de temperaturas, veamos cómo leer todo el contenido del fichero de una sola vez. Para esta operación, Python nos provee, al menos, de dos funciones:

`read()`

Devuelve todo el contenido del fichero como una cadena de texto (`str`):

```
>>> # Podemos obviar 'r' ya que es el modo por defecto!
>>> f = open('files/temps.dat')

>>> f.read()
'23 29\n23 31\n26 34\n23 33\n22 29\n22 28\n22 28\n'
```

>>>

`readlines()`

Devuelve todo el contenido del fichero como una lista (`list`) donde cada elemento es una línea:

```
>>> f = open('files/temps.dat')

>>> f.readlines()
['23 29\n', '23 31\n', '26 34\n', '23 33\n', '22 29\n', '22 28\n', '22 28\n']
```

>>>

Importante

Nótese que, en ambos casos, los saltos de línea `\n` siguen apareciendo en los datos leídos, por lo que habría que «limpiar» estos caracteres. Para ello se recomienda utilizar [las funciones ya vistas de cadenas de texto](#).

Lectura línea a línea

Hay situaciones en las que interesa leer el contenido del fichero línea a línea. Imaginemos un fichero de tamaño considerable (varios GB). Si intentamos leer completamente este fichero de sola una vez podríamos ocupar demasiada RAM y reducir el rendimiento de nuestra máquina.

Es por ello que Python nos ofrece varias aproximaciones a la lectura de ficheros línea a línea. La más usada es iterar sobre el propio *manejador* del fichero, ya que los ficheros son estructuras de datos **iterables**:

```
>>> f = open('files/temps.dat')

>>> for line in f:    # that easy!
...     print(line)
...
23 29

23 31

26 34

23 33
```

>>>

22 29

22 28

22 28

Truco

Igual que pasaba anteriormente, la lectura línea por línea también incluye el **salto de línea** `\n` lo que provoca un «doble espacio» entre cada una de las salidas. Bastaría con aplicar `line.strip()` para eliminarlo.

Lectura de una línea

Hay ocasiones en las que nos interesa leer únicamente una sola línea. Es cierto que esto se puede conseguir mediante la aproximación anterior. Sería algo como:

```
>>> f = open('files/temps.dat')
```

```
>>> for line in f:
...     print(line)
...     break
... 
```

23 29

Pero Python también ofrece la función `readline()` que nos devuelve la siguiente línea del fichero:

```
>>> f = open('files/temps.dat')
```

```
>>> f.readline()
'23 29\n'
```

Es importante señalar que cuando utilizamos la función `readline()` **el «puntero de lectura» se desplaza a la siguiente línea del fichero**, con lo que podemos seguir cargando la información según nos interese:

```
>>> f = open('files/temps.dat')
```

```
>>> # Lectura de las 3 primeras líneas
>>> for _ in range(3):
...     print(f.readline().strip())
... 
```

23 29
23 31
26 34

```
>>> # Lectura de las restantes líneas (4)
>>> for line in f:
...     print(line.strip())
```

```
...
23 33
22 29
22 28
22 28
```

Los ficheros se agotan

Hay que tener en cuenta que, una vez abierto el fichero, **la lectura de su contenido se puede realizar una única vez**. O dicho de otra manera, el iterable que lleva implícito «se agota».

Veamos este escenario con el ejemplo anterior:

```
>>> f = open('files/temps.dat')
>>> for line in f:
...     print(line.strip(), end=' ')
...
23 29 23 31 26 34 23 33 22 29 22 28 22 28
>>> for line in f:
...     print(line.strip(), end=' ')
... # No hay salida!!
```

Esto mismo ocurre si utilizamos funciones como `read()` o `readlines()`.

Advertencia

Por este motivo y también por cuestiones de legibilidad del código, deberíamos abrir un fichero una única vez y realizar todas las operaciones de lectura necesarias, siempre que las circunstancias lo permitan.

Escritura en un fichero

Para escribir texto en un fichero hay que abrir dicho fichero en **modo escritura**. Para ello utilizamos el *argumento adicional* en la función `open()` que indica esta operación:

```
>>> f = open('files/canary-iata.dat', 'w')
```

Nota

Si bien el fichero en sí mismo se crea al abrirlo en modo escritura, la **ruta** hasta ese fichero no. Eso quiere decir que debemos asegurarnos que **las carpetas hasta llegar a dicho fichero existen**. En otro caso obtenemos un error de tipo `FileNotFoundError`.

Ahora ya podemos hacer uso de la función `write()` para enviar contenido al fichero abierto.

Supongamos que queremos volcar el contenido de una lista/tupla en dicho fichero. En este caso partimos de los *códigos IATA* de aeropuertos de las Islas Canarias ².

```

1 >>> canary_iata = ('TFN', 'TFS', 'LPA', 'GMZ', 'VDE', 'SPC', 'ACE', 'FUE')
2
3 >>> for code in canary_iata:
4 ...     f.write(code + '\n')
5 ...
6
7 >>> f.close()

```

Nótese:

Línea 4

Escritura de cada código en el fichero. La función `write()` no incluye el salto de línea por defecto, así que lo añadimos de *manera explícita*.

Línea 7

Cierre del fichero con la función `close()`. Especialmente en el caso de la escritura de ficheros, se recomienda encarecidamente cerrar los ficheros para evitar pérdida de datos.

Advertencia

Siempre que se abre un fichero en **modo escritura** utilizando el argumento `'w'`, el fichero se inicializa, borrando cualquier contenido que pudiera tener.

Otra forma de **escribir la tupla «de una sola vez»** podría ser utilizando la función `join()` con el *salto de línea* como separador:

```

>>> canary_iata = ('TFN', 'TFS', 'LPA', 'GMZ', 'VDE', 'SPC', 'ACE', 'FUE')
>>> f = open('files/canary-iata.dat', 'w')
>>> f.write('\n'.join(canary_iata))
>>> f.close()

```

En el caso de que ya tengamos una **lista (iterable) cuyos elementos tengan el formato de salida que necesitamos** (incluyendo salto de línea si así fuera necesario) podemos utilizar la función `writelines()` que nos ofrece Python.

Siguiendo con el ejemplo anterior, imaginemos un escenario en el que la tupla ya contiene los saltos de línea:

```

>>> canary_iata = ('TFN\n', 'TFS\n', 'LPA\n', 'GMZ\n', 'VDE\n', 'SPC\n', 'ACE\n', 'FUE\n')
>>> f = open('files/canary-iata.dat', 'w')
>>> f.writelines(canary_iata)

```

```
>>> f.close()
```

Truco

Esta aproximación puede ser interesante cuando leemos de un fichero y escribimos en otro ya que las líneas «vienen» con el salto de línea ya incorporado.

Añadido a un fichero

La única diferencia entre añadir información a un fichero y [escribir información en un fichero](#) es el modo de apertura del fichero. En este caso utilizamos 'a' por «append»:

```
>>> f = open('more-data.txt', 'a')
```

>>>

En este caso el fichero `more-data.txt` se abrirá en *modo añadir* con lo que las llamadas a la función `write()` hará que aparezcan nueva información al final del contenido ya existente en dicho fichero.

Usando contextos

Python ofrece [gestores de contexto](#) como una solución para establecer reglas de entrada y salida a un determinado bloque de código.

En el caso que nos ocupa, usaremos la sentencia `with` y el contexto creado se ocupará de cerrar adecuadamente el fichero que hemos abierto, liberando así sus recursos:

```
1  >>> with open('files/temps.dat') as f:
2      ...     for line in f:
3          ...         min_temp, max_temp = line.strip().split()
4          ...         print(min_temp, max_temp)
5      ...
6      23 29
7      23 31
8      26 34
9      23 33
10     22 29
11     22 28
12     22 28
```

>>>

Línea 1

Apertura del fichero en *modo lectura* utilizando el gestor de contexto definido por la palabra reservada `with`.

Línea 2

Lectura del fichero línea a línea utilizando la iteración sobre el *manejador del fichero*.

Línea 3

Limpieza de saltos de línea con `strip()` encadenando la función `split()` para separar las dos temperaturas por el carácter *espacio*. Ver [limpiar una cadena](#) y [dividir una cadena](#).

Línea 4

Imprimir por pantalla la temperatura mínima y la máxima.

Nota

Es una buena práctica usar `with` cuando se manejan ficheros. La ventaja es que el fichero se cierra adecuadamente en cualquier circunstancia, incluso si se produce cualquier **tipo de error**.

Hay que prestar atención a la hora de escribir valores numéricos en un fichero, ya que el método `write()` por defecto espera ver un «string» como argumento:

```
>>> lottery = [43, 21, 99, 18, 37, 99]
```

>>>

```
>>> with open('files/lottery.dat', 'w') as f:
```

```
...     for number in lottery:
```

```
...         f.write(number + '\n')
```

```
...
```

Traceback (most recent call last):

File "<stdin>", line 3, in <module>

TypeError: write() argument must be str, not int

Importante

Para evitar este tipo de **errores**, se debe convertir a `str` aquellos valores que queramos usar con la función `write()` para escribir información en un fichero de texto. Los [f-strings](#) son tu aliado.